Taller Práctico - Transferencia de Conocimiento

El objetivo principal del taller es aprender lo necesario sobre las herramientas básicas aplicables a gran parte de los casos de uso de inteligencia artificial, basados en arquitecturas RAG (Retrieval Augmented Generation). Para ello, entenderemos como funciona la búsqueda semántica y las distintas maneras de ponerla a disposición para nuestros modelos LLm.

1. Despliegue de NIMs

El primer paso, es tener modelos corriendo en nuestra arquitectura, para ello nos dirigimos al catálogo NGC de Nvidia donde podemos encontrar NIM's y contenedores que albergan todo el entorno de ejecución que cada modelo necesita.

Una vez elegido el modelo que deseamos implementar, para este ejercicio usaremos gemma-4:12b, simplemente descargamos la imagen usando el orquestador de preferencia, en este caso, usaremos docker.

```
docker pull nvcr.io/nim/google/gemma-4-26b-a4b-it:2.1.4
```

Una vez descargado, ejecutamos el contenedor:

```
docker run \
    # Flag para que corra en segundo plano
    -d \
    # Le permitimos el uso de GPU
    --gpus=all
    # Establecemos un limite de VRAM (segun el modelo)
    --gpu-memory-utilization 0.5
    # Nombramos el contenedor de manera que podamos detenerlo posteriormente
    --name test-nim
    # Mapeo de puertos (por defecto Triton Inference Server usa el puerto 8000)
    -p 8000:8000
    # Corremos el contenedor en el perfil optimizado
    -e NIM_MODEL_PROFILE="nombre_del_perfile"
    # Indicamos la API key (Developer Program o AI Enterprise)
    -e NGC_API_KEY="your_api_key"
    # Imagen y TAG
    nvcr.io/nim/google/gemma-4-26b-a4b-it:2.1.4
```

Si no tenemos identificado el perfil, podemos verlos usando la siguiente flag:

```
docker run --rm --gpus=all nvcr.io/nim/google/gemma-4-26b-a4b-it:2.1.4 \
    # Esto nos retorna la lista de perfiles disponibles
    list-model-profiles
```

Ahora repetimos este procedimiento para el nim que genera los embeddings para la búsqueda semántica, Nemotron 3 Embed 1B

```
docker pull nvcr.io/nim/nvidia/nemotron-3-embed-1b:latest
```

2. RAG (Retrieval Augmented Generation)

Es un patrón de arquitectura en Inteligencia Artificial que conecta un modelo de lenguaje (LLM) con fuentes externas de información en tiempo real para responder con precisión y sin inventar datos.

¿Por qué es mejor así?

Los modelos tienen muchísimo conocimiento, pero son un océano de 1 metro de profundidad, saben casi sobre todo lo que preguntes, sin embargo solo saben lo básico, el problema, es que para nuestros casos de uso necesitan que en ciertas partes de ese océano de conocimiento, el modelo tenga kilómetros de profundidad.

El Límite de Contexto

Aunque con ayuda de un `system prompt` o prompt del sistema, podemos brindar la información necesaria y los conocimientos necesarios para nuestro caso de uso, las capas de atención sufren intentando retener mucha información, y cualquier minúsculo detalle se pierde en la medida en que nuestra conversación crece, esto sucede porque las capas de atención suelen centrarse en lo más reciente de la conversación.

La Solución

Traer la información pertinente para resolver una solicitud o responder a una pregunta, en la medida en que el contexto de la conversación lo va requiriendo, como analogía, es como fragmentar el system prompt e irlo dosificando en base a la conversación.

3. ¿Cómo funciona el RAG?

Embeddings: Los modelos de lenguaje no procesan la conversación tal cual nosotros la escribimos, en el fondo esta realizando millones de operaciones matemáticas en base a una representación vectorial de la conversación. Casi que podemos asociar palabras con tokens, con ligeros matices, y esos tokens son números flotantes, así bien, antes de que el modelo

procese la conversación hay un `tokenizer` que convierte la conversación en dicha representación numérica. Cuando hablamos de Embeddings, hablamos precisamente de esa representación numérica de la información proporcionada, que puede ser texto, imagen, audio, entre otros, pero no es tan simple como un tokenizador, en este caso las representaciones vectoriales están distribuidas de manera que conceptos similares se agrupan, obteniendo que por ejemplo, la palabra o el token de **gato** se encuentre cerca a **animales** y la palabra **carro** se encuentre lejos de estas.

Bases de datos vectoriales: Una vez tenemos esa representación vectorial de la información, podemos almacenarla en bases de datos vectoriales, lo que nos premite posteriormente, realizar búsqueda semántica, es decir, en lugar de buscar por criterios exactos, podemos buscar por relaciones semánticas, de manera que al buscar **perro** los resultados por probabilidad serán frases que contienen términos relacionados como: ladrar, caninos, mordedura, etc.

¿Cual es el flujo de trabajo? El RAG es una base de datos vectorial en la cual depositamos los fragmentos de uno o varios documentos, de manera que el agente pueda recuperar la información que tiene estrecha relación con los requerimientos del usuario final.

4. RAG - Direct Injection

En esta metodología la entrada del usuario pasa primero por la búsqueda semántica y los resultados se adjuntan al contexto del agente antes de que genere una respuesta, en pocas palabras, el agente toma decisiones y responde **basado en datos**.

```
[Pregunta del usuario]
    v
[Búsqueda semántica]
    v
[Contexto en base a resultados]
    v
[Respuesta del agente]
```

5. Implementación de un RAG

Existen múltiples opciones de bases de datos vectoriales open source como: Milvus, Chroma DB, Qdrant, FAISS, y opciones licenciadas como las que ofrencen MongoDB, PostrgreSQL y Redis, cada una con sus ventajas y desventajas, SANOLIVAR aunque no ha experimentado todas las opciones disponibles, por experiencia recomienda el uso de FAISS pese a sus limitaciones, y la razón es su rendimiento, ya que ofrece búsqueda e indexación muy rápida en comparación con otras opciones probadas, incluso usando la versión que no usa GPU.

La debilidad de **FAISS** (entre otras opciones) es que no almacena metadatos, por lo cual, la información original debe se respaldada en otra base de datos, como MongoDB o SQL. Por lo cual, para construir este componente, usaremos una imagen oficial de mongo, y para no generar microservicios de uno de los microservicios de la arquitectura, sobre este contenedor instalaremos FAISS y expondremos la arquitectura usando un servidor flask.

6. Agentic RAG

En esta metodología cambia la manera en que se injectan la información, en lugar de una injección directa, es el agente quien decide cuando consultar una base de conocimientos, de esta manera no solo se ahorra contexto (se puede saturar de información innecesaria), sino también evitamos que el agente tenga alucinaciones generadas por injección de información irrelevante. No debemos verla como una mejor manera del RAG convencional, sino como una alternativa más eficiente para casos de uso particulares.

En esta metodología es importante describir las bases de conocimiento y su contenido de manera muy breve, de manera que el agente sepa en donde debe buscar dependiendo de la información disponible y como esté clasificada. Además esto permite que podamos segmentar la información de acuerdo a criterios de clasificación, ya sea por gobernanza de datos, disciplina cognitiva, entre otras. Otra ventaja de esta metodología es que el agente puede refinar los criterios de búsqueda, usando palabras clave mucho más eficientes.

```
[Pregunta del usuario]
    v
[Agente decide que y donde buscar] -> [conocimientos_1] [conocimientos_2] ...
    v
[Respuesta del agente]
```

7. Conclusiones

Dependiendo del caso de uso podemos aplicar una u otra metodología, también podemos segmentar las bases de conocimiento según los criterios esperados, como gobernanza de datos, creando agentes segun los roles empresariales con acceso a información específica de manera que la información sensible solo sea accesible a quien corresponde, o por disciplinas, teniendo arquitecturas multiagente en los que cada agente tiene información específica de manera que su contexto no se sature con múltiples disciplinas.

Dejar de usar system prompts extremadamente largos que por el mismo funcionamiento de los modelos se va a perder en la medida en que la conversación se hace larga, y en su lugar tener información granular que llegue de forma dinámica a la conversación, es una manera

más controlada y eficiente de brindar información a un agente para que cumpla con los requerimientos del usuario final.

Por cuestiones de seguridad aunque cueste más una implementación, es mejor tener múltiples agentes con información restringida, que un agente universal con acceso a todo tipo de información, además vale recalcar que aunque en el ejercicio por cuestiones de tiempo no se implementó, Nvidia GuardRails es una capa extra que permite que la información delicada se mantenga segura por más que el usuario a través de prompt engineering, intente convencer a un agente de que le brinde información, ya que los modelos del lenguaje, por si solos, fácilmente pueden cambiar de comportamiento si alguien conocedor intenta manipular sus directrices.